

Software Aging in LLM-Generated Applications: Runtime Evidence, Static Analysis, and Human-Written Comparisons

Online Annex: Detailed Analysis of Runtime Aging Evidence

César Santos, Michele Vitagliano, Roberto Natella, Ermeson Andrade

I. SOFTWARE AGING ANALYSIS (RQ1)

To investigate **RQ1**, we analyzed whether LLM-generated applications exhibit software aging symptoms during long-duration execution. We first examine memory usage, measured as the total RAM consumption reported by the operating system. Then, we analyze response time and throughput as complementary performance indicators collected from the client side using Apache JMeter. Since software aging may manifest through progressive resource consumption, increasing latency, or reduced processing capacity, these metrics were analyzed both visually and statistically using the Mann–Kendall test and Sen’s slope estimator.

A. Memory usage

Figure 1 shows the memory usage behavior of the *Credit Card application* across the three programming languages. In Figure 1 (a), the JavaScript implementation presents a noticeable increase in memory usage after approximately 16 hours of execution. After this growth phase, memory consumption remains at a higher but relatively stable level, with small oscillations over time. This behavior suggests a step-like increase in resource consumption rather than a continuous linear growth pattern.

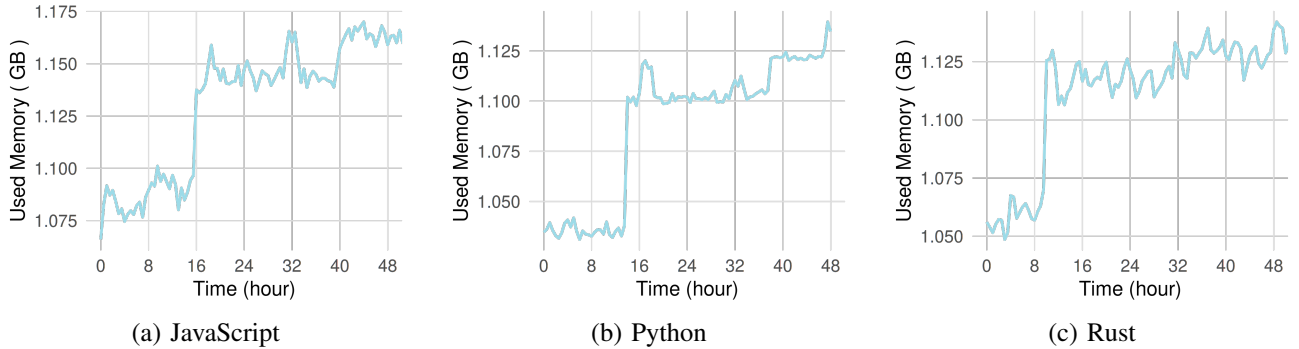


Fig. 1: Memory usage for the Credit Card application across generation-and-execution environments.

In Figure 1 (b), the Python implementation exhibits a sharp increase in memory usage around 15 hours after the beginning of the experiment. Unlike the JavaScript implementation, this increase is followed by a continued upward tendency with fluctuations, suggesting progressive resource accumulation during execution. Figure 1 (c), corresponding to the Rust implementation, shows a gradual increase from the beginning of the experiment, followed by a more pronounced growth around hour 8 and intermittent peaks throughout the remaining execution period. The three implementations of the *Credit Card application* exhibit memory growth during the 48-hour execution, but the shape and intensity of this growth differ across languages. This behavior provides preliminary visual evidence of memory-related software aging symptoms in this application scenario.

To confirm these observations, we performed a statistical analysis using the Mann–Kendall test and Sen’s slope estimation. The results are summarized in Table I. All p-values are approximately zero, indicating statistically significant upward trends in memory usage. Among the analyzed languages, Python exhibits the highest slope, suggesting the fastest memory growth rate, followed by JavaScript and Rust. These findings provide statistical evidence of memory-related aging symptoms in all Credit Card implementations, with varying intensity across generation-and-execution environments.

Figure 2 presents the memory usage behavior of the *Image Converter application*. In Figure 2 (a), the JavaScript implementation exhibits a clear upward trajectory throughout the execution period, with frequent oscillations and memory usage spikes. This indicates increasing resource consumption under the image-processing workload. In Figure 2 (b), the Python implementation shows a sharp increase in memory usage at the beginning of the execution, followed by a gradual and steady growth pattern until the end of the experiment. This behavior also suggests progressive resource accumulation over time. In

Language	p-value	Slope (GB/hour)
JavaScript	≈ 0	1.855×10^{-3}
Python	≈ 0	2.154×10^{-3}
Rust	≈ 0	1.400×10^{-3}

TABLE I: Memory usage trends for the Credit Card application across languages.

contrast, Figure 2 (c), corresponding to the Rust implementation, shows an initial growth phase followed by a reduction in memory usage and a relatively stable pattern during the remainder of the execution.

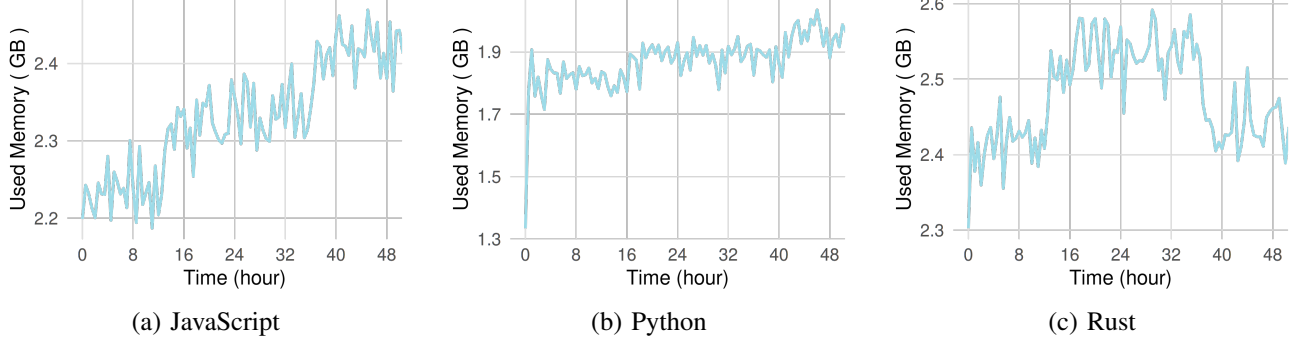


Fig. 2: Memory usage for the Image Converter application across generation-and-execution environments.

The statistical results in Table II confirm the visual observations. JavaScript and Python present statistically significant increasing trends, with p-values approximately equal to zero. JavaScript shows the highest slope, followed by Python, indicating that these two implementations exhibit clear memory growth during the experiment. In contrast, the Rust implementation presents a p-value of 0.205, indicating that its observed memory trend is not statistically significant. Therefore, in this specific workload, the Rust implementation does not present clear evidence of progressive memory growth, unlike the JavaScript and Python implementations.

Language	p-value	Slope (GB/hour)
JavaScript	≈ 0	4.410×10^{-3}
Python	≈ 0	3.744×10^{-3}
Rust	0.205	0.396×10^{-3}

TABLE II: Memory usage trends for the Image Converter application across languages.

Figure 3 shows the memory usage behavior of the *Monitor application*. In Figure 3 (a), the JavaScript implementation remains initially stable and then presents a noticeable increase around hour 10. After this increase, memory usage remains at a higher level, with small fluctuations, indicating a step-like growth pattern. In Figure 3 (b), the Python implementation shows a similar initial behavior, followed by a more pronounced increase and subsequent gradual growth with minor oscillations. This suggests sustained memory accumulation over time. Figure 3 (c), corresponding to the Rust implementation, presents an initial increase, a stabilization period, and a later growth phase, suggesting a controlled but persistent upward tendency.

As shown in Table III, all Monitor implementations present statistically significant upward trends in memory usage. Python has the highest slope, followed by Rust and JavaScript. These results indicate that memory-related aging symptoms are present in the three implementations of the Monitor application, although the intensity of memory growth differs across languages.

Figure 4 presents the memory usage behavior of the *Uptime application*. In Figure 4 (a), the JavaScript implementation shows an initially stable memory usage followed by a sharp increase around hour 18. After this increase, memory usage slightly decreases and then stabilizes with minor fluctuations. In Figure 4 (b), the Python implementation grows in distinct steps throughout the execution, resulting in a higher memory level toward the end of the experiment. Figure 4 (c), corresponding to the Rust implementation, exhibits an initial increase followed by a relatively stable phase and a later gradual growth.

The statistical results in Table IV show statistically significant upward memory trends for all Uptime implementations. Python presents the highest slope, followed by Rust and JavaScript. These findings indicate memory-related aging symptoms in the three implementations of the Uptime application.

Memory usage was the most consistent indicator of potential software aging symptoms across the experiments. Eleven of the twelve application-language combinations presented statistically significant upward trends, with the Rust implementation of the Image Converter application being the only non-significant case. The largest memory growth was observed in the Image Converter application for JavaScript, with a slope of 4.410×10^{-3} , followed by Python in the same application, with 3.744×10^{-3} . This suggests that the image-processing and file-handling scenario imposed the strongest memory pressure on

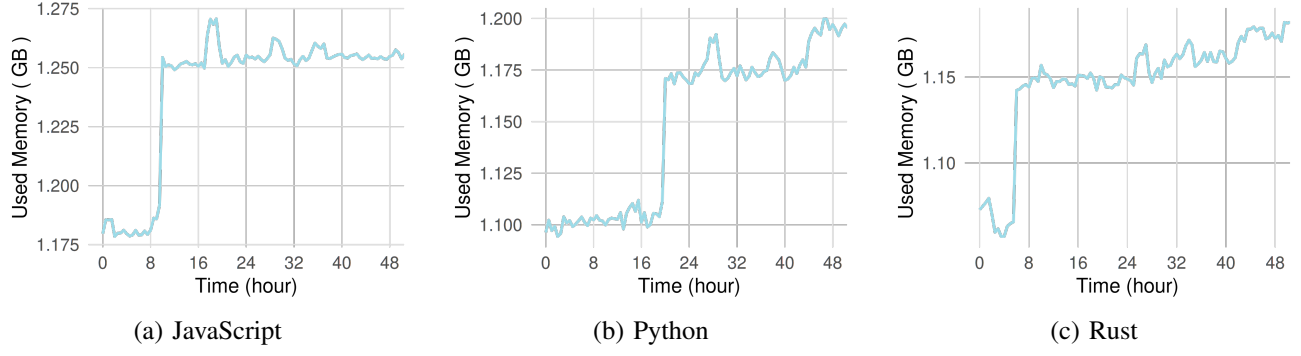


Fig. 3: Memory usage for the Monitor application across programming languages.

Language	p-value	Slope (GB/hour)
JavaScript	≈ 0	1.332×10^{-3}
Python	≈ 0	2.324×10^{-3}
Rust	≈ 0	1.612×10^{-3}

TABLE III: Memory usage trends for the Monitor application across languages.

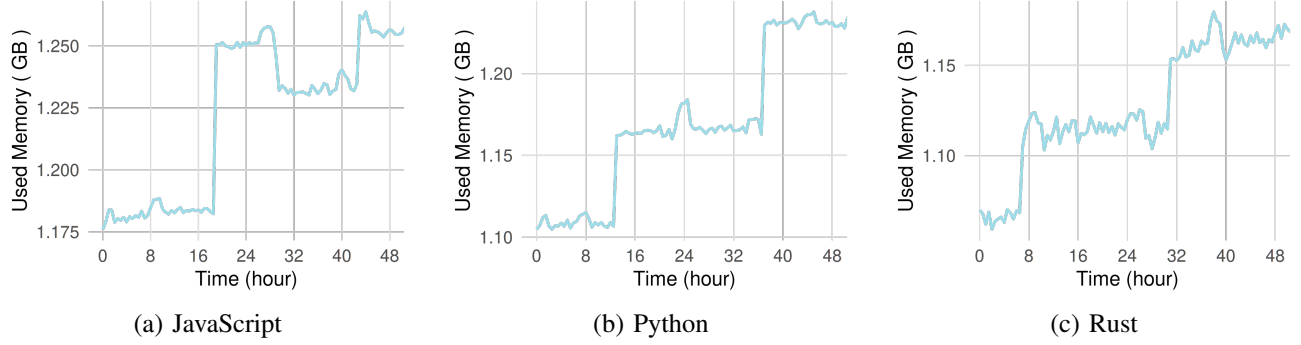


Fig. 4: Memory usage for the Uptime application across programming languages.

Language	p-value	Slope (GB/hour)
JavaScript	≈ 0	1.732×10^{-3}
Python	≈ 0	2.809×10^{-3}
Rust	≈ 0	2.066×10^{-3}

TABLE IV: Memory usage trends for the Uptime application across languages.

the generated Node.js/Express and FastAPI implementations. By contrast, Rust showed a much lower and non-significant slope in this scenario. This may be related to Rust’s memory-management model, in which temporary data are typically released when they leave their execution scope, and to Actix Web’s request-handling model, where request-scoped data are usually released after each handler completes.

Across applications, Python presented the highest memory-growth slope in three scenarios: Credit Card, Monitor, and Uptime. This pattern suggests that the ChatGPT-generated FastAPI implementations were more prone to gradual memory accumulation under sustained execution in these cases. JavaScript showed the highest slope only in the Image Converter scenario, indicating that the Bolt-generated Node.js/Express implementation was particularly sensitive to that workload. Rust generally exhibited lower slopes than Python and JavaScript, but it still showed statistically significant memory growth in Credit Card, Monitor, and Uptime. Therefore, the results do not indicate that any language or generation platform is immune to memory-related aging symptoms. Instead, memory growth appears to depend on the interaction between application behavior, runtime environment, framework, and generated-code structure.

B. Response time

In addition to memory consumption, response time is a key performance metric for identifying potential symptoms of software aging. While memory usage reflects resource accumulation over time, response time provides a direct indication of performance degradation from the user’s perspective. Unlike memory usage, response time may be strongly affected by

transient events, such as short-term contention, I/O delays, or isolated request failures. Therefore, response time trends are interpreted together with the visual behavior of the time series.

Figure 5 shows the response time behavior of the *Credit Card application*. The JavaScript implementation, shown in Figure 5 (a), remains mostly stable throughout the execution, although intermittent spikes are observed. These peaks suggest temporary performance degradation events, possibly caused by short-term resource contention or transient execution conditions. The Python implementation, shown in Figure 5 (b), presents a sharper increase around hour 20, followed by fluctuations and partial stabilization. The Rust implementation, shown in Figure 5 (c), exhibits low response time variance during most of the execution, with a noticeable spike near the end of the experiment.

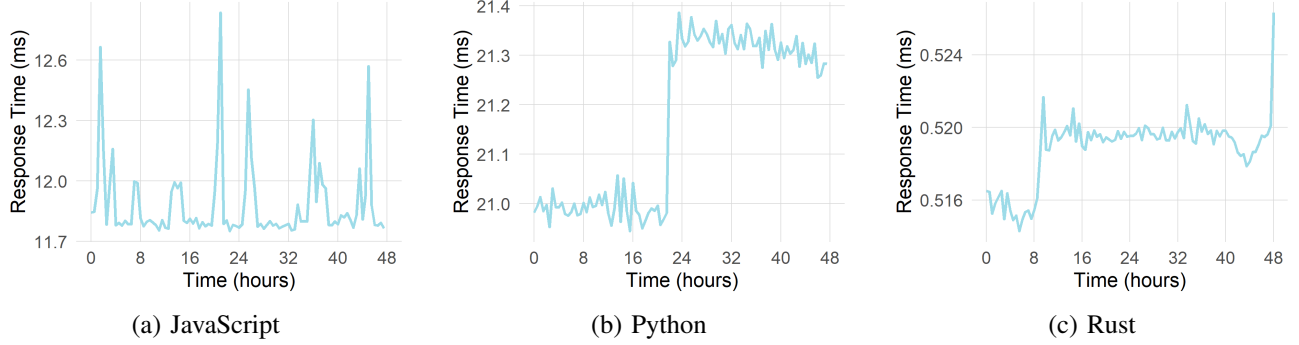


Fig. 5: Response time for the Credit Card application across programming languages.

Table V shows that JavaScript does not present a statistically significant response time trend, with a p-value of 0.772. In contrast, Python and Rust present statistically significant positive slopes. The Python implementation exhibits the highest slope among the three languages, indicating a clearer increase in response time over time. Although the Rust trend is statistically significant, its slope is very small, suggesting that the magnitude of latency growth is limited.

Language	p-value	Slope (ms/hour)
JavaScript	0.772	-4.154×10^{-4}
Python	≈ 0	9.831×10^{-3}
Rust	≈ 0	0.072×10^{-3}

TABLE V: Response time trends for the Credit Card application across languages.

Figure 6 presents the response time behavior of the *Image Converter application*. In Figure 6 (a), the JavaScript implementation shows highly variable response times with frequent fluctuations and a slight increasing tendency. The Python implementation, shown in Figure 6 (b), also presents high variability, but without a clear monotonic pattern. The Rust implementation, shown in Figure 6 (c), remains relatively stable for most of the execution but exhibits a large spike around hour 35, indicating a severe but isolated performance anomaly.

The statistical results in Table VI show that JavaScript presents a statistically significant positive trend, with a p-value of 0.032. Python does not present a statistically significant trend, with a p-value of 0.343, indicating no clear monotonic response time degradation. Rust presents a statistically significant result, with a p-value of 1.74×10^{-14} and a very large positive slope. However, this result should be interpreted with caution, since the time series is strongly affected by an extreme response-time spike rather than by a smooth monotonic degradation pattern.

Language	p-value	Slope (ms/hour)
JavaScript	0.032	0.4136
Python	0.343	-0.688
Rust	1.744×10^{-14}	71.027

TABLE VI: Response time trends for the Image Converter application across languages.

Figure 7 shows the response time behavior of the *Monitor application*. In Figure 7 (a), the JavaScript implementation shows an initial slight decrease in response time followed by a gradual increase starting around hour 25. After this point, response time stabilizes at a higher level, indicating a step-like degradation pattern. The Python implementation, shown in Figure 7 (b), exhibits high variability with frequent spikes but no clear upward or downward trend. The Rust implementation, shown in Figure 7 (c), remains mostly stable with small oscillations, although a mild increase and a spike near the end of the experiment can be observed.

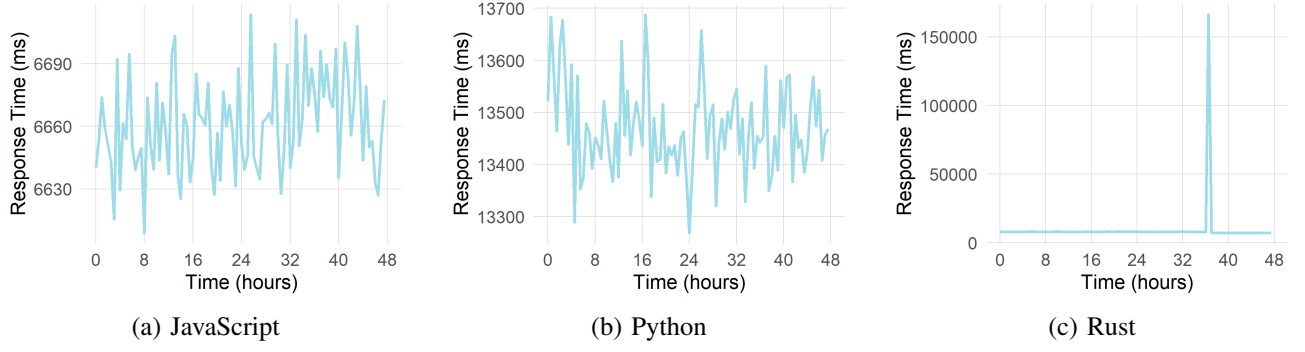


Fig. 6: Response time for the Image Converter application across programming languages.

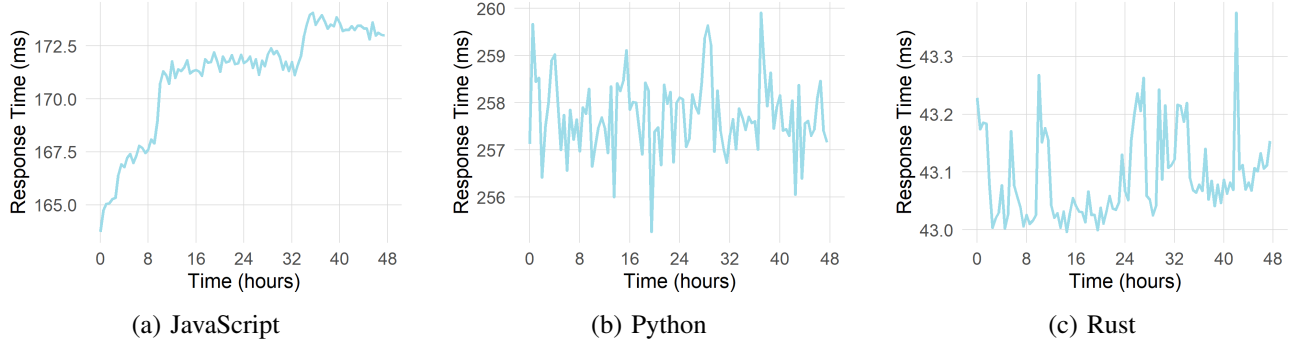


Fig. 7: Response time for the Monitor application across programming languages.

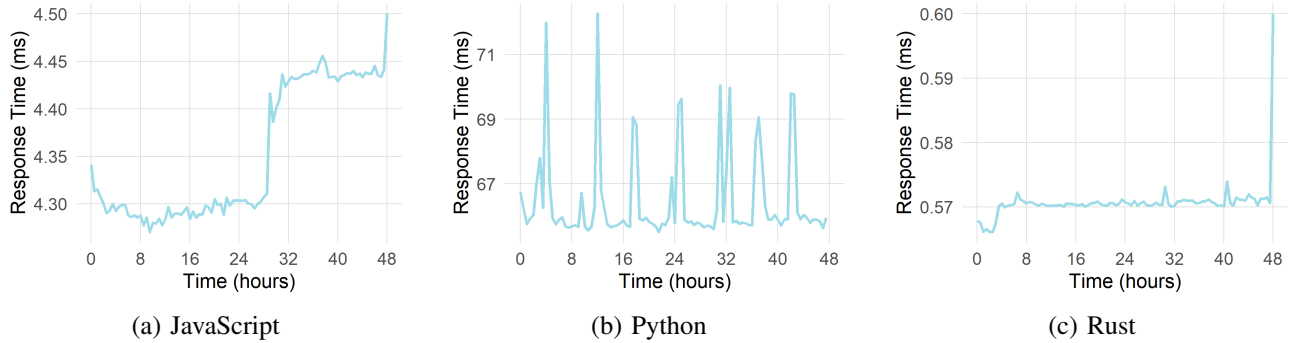


Fig. 8: Response time for the Uptime application across programming languages.

As shown in Table VII, JavaScript and Rust present statistically significant positive trends in response time, while Python does not present a significant trend. The JavaScript implementation shows the highest slope, indicating a stronger latency increase over time. Rust also presents a significant trend, but with a much smaller slope.

Language	p-value	Slope (ms/hour)
JavaScript	≈ 0	0.154
Python	0.825	0.034×10^{-3}
Rust	1.192×10^{-07}	1.157×10^{-3}

TABLE VII: Response time trends for the Monitor application across languages.

Figure 8 presents the response time behavior of the *Uptime* application. In Figure 8 (a), the JavaScript implementation shows an initial decrease in response time followed by a gradual increase after approximately hour 25, leading to a higher and relatively stable response time level. In Figure 8 (b), the Python implementation is characterized by frequent spikes throughout the execution period, but without a clear progressive degradation pattern. Figure 8 (c), corresponding to the Rust implementation, remains mostly stable during most of the experiment but presents a sharp increase near the end.

Table VIII shows that JavaScript and Rust present statistically significant positive response time trends, while Python does

not show a significant trend. The Python implementation has a negative slope, but the non-significant p-value indicates that this should not be interpreted as evidence of systematic performance improvement. Overall, response time behavior is less consistent than memory usage, with several cases showing transient spikes or workload-specific instability rather than smooth monotonic degradation.

Language	p-value	Slope (ms/hour)
JavaScript	≈ 0	4.113×10^{-3}
Python	0.872	-2.999×10^{-3}
Rust	≈ 0	0.061×10^{-3}

TABLE VIII: Response time trends for the Uptime application across languages.

Response time presented a less uniform degradation pattern than memory usage. JavaScript showed the clearest latency increase in the Image Converter and Monitor applications, while Python exhibited a significant trend only in Credit Card. Rust showed statistically significant trends in several cases, but the slopes were generally small, except for Image Converter, where the result appears to be influenced by an extreme latency spike rather than by smooth degradation. These findings indicate that response time is more sensitive to transient execution events and application-specific behavior, especially in scenarios involving image processing and file handling. Thus, response time should be interpreted as complementary evidence of aging symptoms, not as a standalone indicator of long-term degradation.

C. Throughput

Throughput was also analyzed to complement the response time results. Since throughput measures the number of requests processed per second, decreasing trends may indicate loss of processing capacity over time and can reinforce evidence of performance degradation. However, throughput and response time capture different aspects of performance behavior: response time reflects the latency perceived by individual requests, while throughput reflects the sustained processing capacity of the system.

Figure 9 shows the throughput behavior of the *Credit Card application*. The JavaScript implementation remains mostly stable during execution, with only short drops. The Python implementation maintains a high throughput during most of the experiment but presents a sharper decrease near the end. The Rust implementation shows a much higher throughput level than the other languages, but also exhibits a decreasing tendency over time.

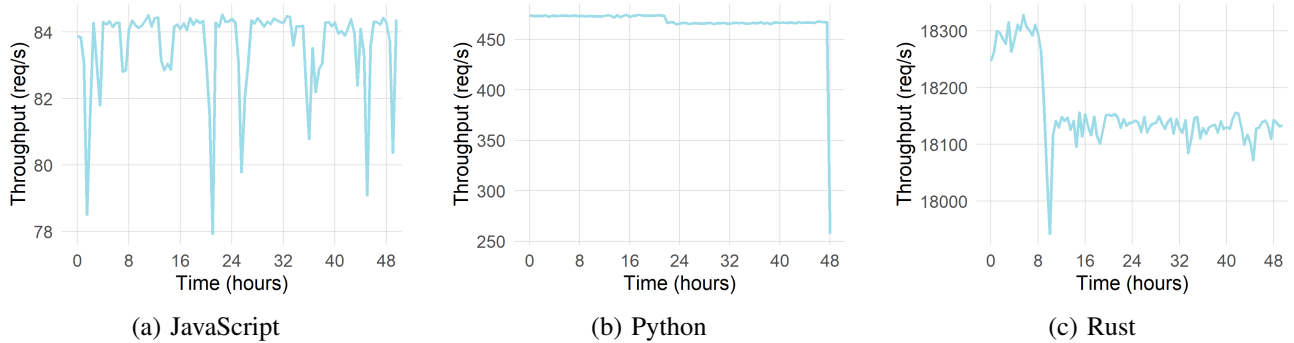


Fig. 9: Throughput for the Credit Card application across programming languages.

The statistical results in Table IX indicate that JavaScript does not present a statistically significant throughput trend. In contrast, Python and Rust present statistically significant negative slopes, suggesting throughput degradation over time. Although Rust achieves the highest throughput level, its negative slope indicates a progressive reduction in processing capacity during the experiment.

Language	p-value	Slope (requests/s)
JavaScript	0.999	-0.584×10^{-3}
Python	9.62×10^{-21}	-0.316
Rust	3.38×10^{-17}	-2.754

TABLE IX: Throughput trends for the Credit Card application across languages.

Figure 10 presents the throughput behavior of the *Image Converter application*. The three implementations operate at lower throughput levels than the other applications, reflecting the heavier nature of the image-processing workload. The JavaScript implementation shows small fluctuations around a stable throughput level, with a slight downward tendency. The Python

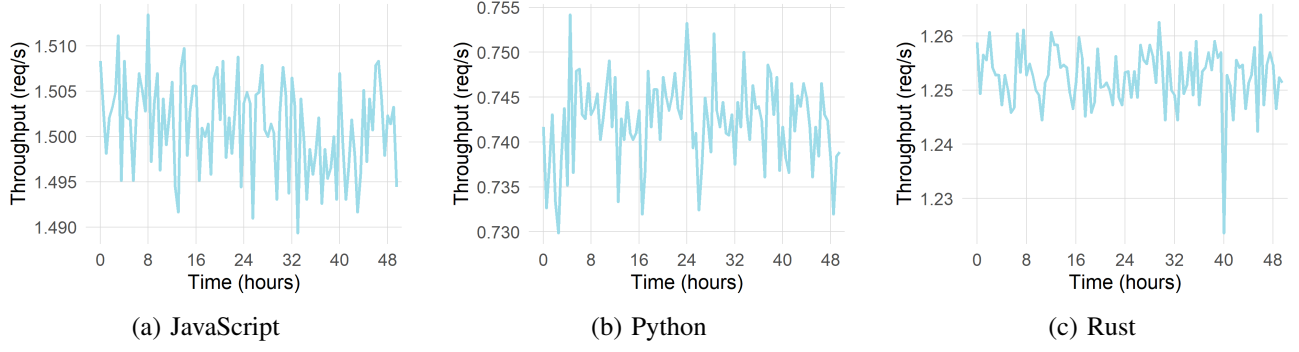


Fig. 10: Throughput for the Image Converter application across programming languages.

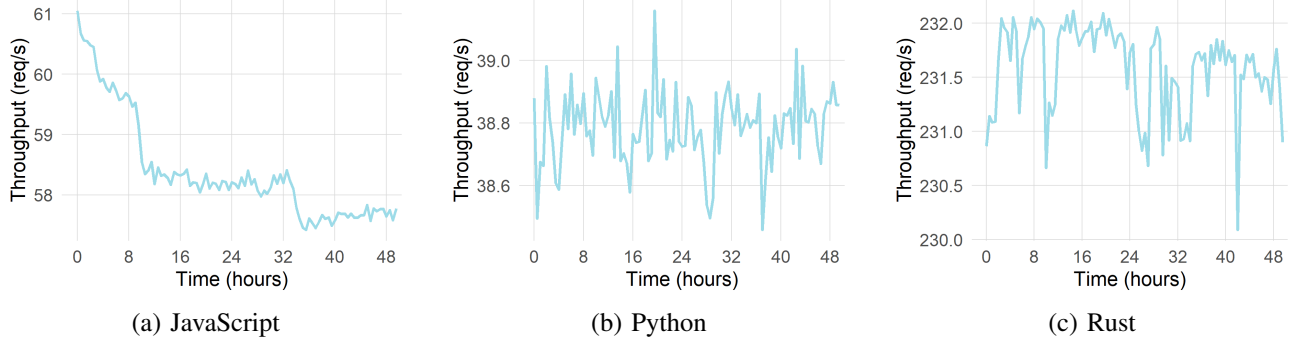


Fig. 11: Throughput for the Monitor application across programming languages.

implementation also presents fluctuations but without a clear long-term degradation pattern. The Rust implementation remains mostly stable throughout the execution, despite isolated drops.

As shown in Table X, JavaScript presents a statistically significant negative trend, although the magnitude of the slope is very small. Python and Rust do not present statistically significant throughput trends, suggesting that their throughput remained relatively stable during the experiment.

Language	p-value	Slope (requests/s)
JavaScript	0.042	-8.959×10^{-05}
Python	0.849	4.2187×10^{-06}
Rust	0.967	-2.826×10^{-05}

TABLE X: Throughput trends for the Image Converter application across languages.

Figure 11 shows the throughput behavior of the *Monitor application*. In Figure 11 (a), the JavaScript implementation presents a clear throughput reduction during the first hours, followed by a lower stable level and another decrease after approximately 32 hours. The Python implementation, shown in Figure 11 (b), presents small oscillations without a clear degradation pattern. The Rust implementation, shown in Figure 11 (c), maintains a higher throughput level than JavaScript and Python but presents visible drops and an overall decreasing tendency.

The statistical results in Table XI show that JavaScript and Rust present statistically significant negative throughput trends, indicating loss of processing capacity over time. Python, however, presents a non-significant p-value, suggesting no consistent throughput degradation during the experiment.

Language	p-value	Slope (requests/s)
JavaScript	≈ 0	-0.050
Python	0.393	0.409×10^{-3}
Rust	0.293×10^{-07}	-6.297×10^{-3}

TABLE XI: Throughput trends for the Monitor application across languages.

Figure 12 shows the throughput behavior of the *Uptime application*. In Figure 12 (a), the JavaScript implementation initially presents a high and slightly increasing throughput, followed by a sharp reduction after approximately 30 hours of execution. After this drop, the throughput remains at a lower and relatively stable level, suggesting a loss of processing capacity during

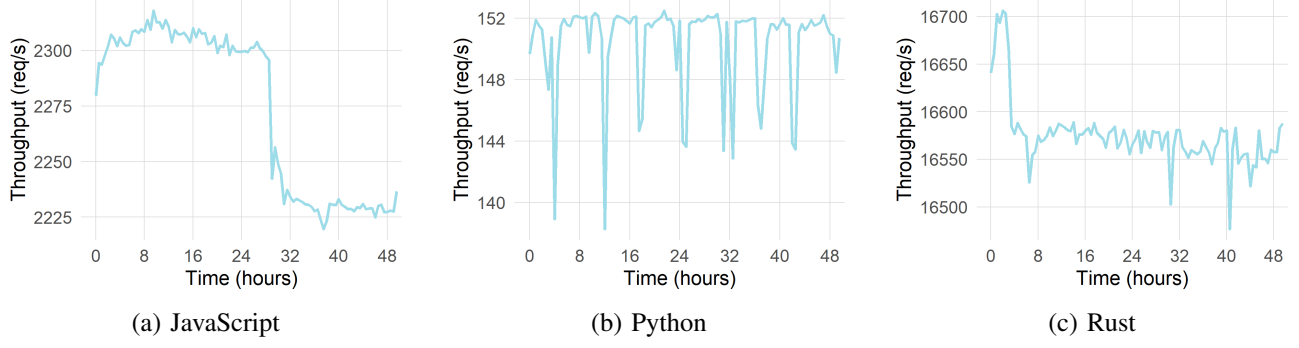


Fig. 12: Throughput for the Uptime application across generation-and-execution environments.

the experiment. The Python implementation, shown in Figure 12 (b), remains mostly stable around its average throughput, although several isolated drops occur throughout the execution. These fluctuations do not indicate a clear long-term degradation pattern. The Rust implementation, shown in Figure 12 (c), starts with a very high throughput, followed by an early decrease and a lower stable level with intermittent drops, suggesting a significant throughput reduction over time.

The statistical results in Table XII confirm the visual behavior observed in Figure 12. JavaScript and Rust present statistically significant negative throughput trends, indicating a reduction in processing capacity over time. In contrast, Python presents a non-significant p-value of 0.383, suggesting that its throughput remained relatively stable despite isolated fluctuations. These results indicate that, for the *Uptime application*, throughput degradation was observed in JavaScript and Rust, but not in Python.

Language	p-value	Slope (requests/s)
JavaScript	≈ 0	-2.186
Python	0.383	5.535×10^{-3}
Rust	6.460×10^{-20}	-1.340

TABLE XII: Throughput trends for the Uptime application across languages.

The throughput results indicate that loss of processing capacity was concentrated in specific application-language combinations rather than uniformly observed across all generated services. Significant negative trends were observed in Python and Rust for the *Credit Card application*, JavaScript and Rust for the *Monitor application*, and JavaScript and Rust for the *Uptime application*. The strongest reductions occurred in Rust for *Credit Card*, with a slope of -2.754, JavaScript for *Uptime*, with -2.186, and Rust for *Uptime*, with -1.340. Although Rust achieved the highest throughput levels in several scenarios, its negative slopes show that high initial processing capacity did not prevent degradation during prolonged execution. Python remained statistically stable in most scenarios, except for *Credit Card*, while JavaScript showed degradation mainly in *Monitor* and *Uptime*. These findings complement the response time analysis by showing that latency variability does not always translate into sustained throughput loss, and that throughput should be interpreted together with memory usage and response time when assessing software aging symptoms in LLM-generated applications.